

SwiftyRing

SISTEMA DE TIMBRE ESCOLAR INTELIGENTE

SwiftyRing

Documento de requerimientos · v4.0

Aplicación móvil Flutter + módulo físico ESP32 · Backend propio con Vapor (Swift) + PostgreSQL. Especificación funcional y técnica del sistema.

CLIENTE	DESARROLLADO POR	VERSIÓN
I.E. Santa Sofía — Dosquebradas, Colombia	Swifty Devs	4.0
FECHA	**PLATAFORMA**	**CAMBIOS V4.0**
Julio de 2026	Flutter · Vapor · PostgreSQL · ESP32	Backend migrado de FastAPI/Python a Vapor (Swift).

CONTENIDO

Índice

01 Visión general

02 Arquitectura del sistema

03 Requerimientos funcionales

- 3.1 Gestión de usuarios y autenticación
- 3.2 Módulo de horarios
- 3.3 Timbre manual
- 3.4 Módulo de detección de sismos
- 3.5 Pantalla OLED en módulo físico
- 3.6 Operación offline
- 3.7 Módulo de profesores y notificaciones de clase

04 Requerimientos no funcionales

05 Stack tecnológico

06 Plan de desarrollo por fases

07 Entregables esperados

01

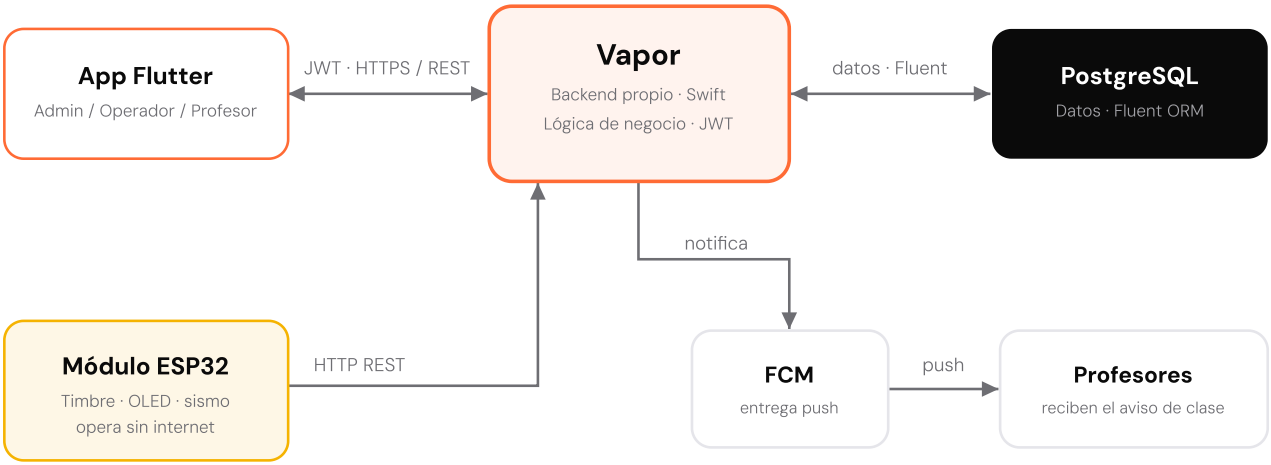
Visión general

Sistema integrado de control de timbre escolar compuesto por una aplicación móvil Flutter y un módulo físico basado en ESP32. El backend es propio: **Vapor + PostgreSQL** alojado en un servidor dedicado, lo que garantiza control total sobre los datos, independencia de plataformas externas y escalabilidad a largo plazo. Las notificaciones push se entregan mediante FCM (Firebase Cloud Messaging) únicamente como canal de entrega, sin depender de ningún otro servicio de Firebase.

02

Arquitectura del sistema

El **backend Vapor** es el centro del sistema: la app y el módulo ESP32 se comunican con él, guarda todo en PostgreSQL y despacha las notificaciones a los profesores por FCM.



El módulo ESP32 llama a Vapor cada vez que el timbre suena —automático, manual o sísmico— y opera de forma autónoma aunque pierda la conexión.

03

Requerimientos funcionales

3.1 · Gestión de usuarios y autenticación

ID	REQUERIMIENTO
RF-01	El sistema contará con autenticación propia mediante JWT (JSON Web Tokens) gestionada desde el backend **Vapor**.
RF-02	Existirá una cuenta de Administrador Principal (Doña Andrea) creada en la inicialización del sistema, no eliminable.
RF-03	El administrador principal podrá crear, editar, suspender y eliminar usuarios secundarios.
RF-04	El sistema manejará tres roles: Administrador, Operador y Profesor.
RF-05	Los operadores podrán consultar horarios y activar el timbre manual, pero no modificar horarios ni gestionar usuarios.
RF-06	El administrador podrá asignar y revocar permisos individuales por usuario.

3.2 · Módulo de horarios

ID	REQUERIMIENTO
RF-07	El sistema permitirá definir un horario habitual semanal con múltiples timbres por día.
RF-08	Cada timbre tendrá: hora, días activos (L–S), etiqueta descriptiva (ej. Inicio de clases) y duración del sonido.
RF-09	El administrador podrá crear, editar y eliminar timbres del horario habitual.
RF-10	El sistema soportará horarios especiales que suspenden temporalmente el horario habitual.
RF-11	Al activar un horario especial, se habilitará un date picker para definir el rango de días de vigencia.
RF-12	Al vencer el rango del horario especial, el sistema retomará automáticamente el horario habitual.
RF-13	Solo el administrador podrá activar, modificar y desactivar horarios especiales.

3.3 · Timbre manual

ID	REQUERIMIENTO
RF-14	La app contará con un botón de timbre manual visible en la pantalla principal.
RF-15	El módulo ESP32 contará con un botón físico de timbre manual.
RF-16	Ambos botones ejecutarán el timbre de forma inmediata, independientemente del horario activo.
RF-17	Cada activación manual quedará registrada en PostgreSQL con usuario, timestamp y origen (app / físico).

3.4 · Módulo de detección de sismos

ID	REQUERIMIENTO
RF-18	El ESP32 contará con un acelerómetro (MPU-6050) para detección de vibraciones sísmicas.
RF-19	El algoritmo de detección deberá distinguir entre sismos y vibraciones de vehículos, usando análisis de frecuencia y duración de la vibración.
RF-20	Un evento sísmico confirmado enviará una notificación push FCM al dispositivo del administrador.
RF-21	La notificación incluirá: hora del evento, intensidad estimada (en g) y duración de la vibración.
RF-22	Al confirmarse un evento sísmico, el módulo ESP32 activará el timbre de forma inmediata y autónoma. Simultáneamente **Vapor** enviará la notificación push al administrador como registro del evento.
RF-23	El historial de eventos sísmicos detectados será consultable desde la app.

3.5 · Pantalla OLED en módulo físico

ID	REQUERIMIENTO
RF-24	El módulo incluirá una pantalla OLED de 1.3" (SH1106).
RF-25	La pantalla mostrará en tiempo real: hora actual, nombre del próximo timbre y tiempo restante.
RF-26	Al activarse el timbre, la pantalla mostrará un indicador visual de timbre activo.
RF-27	Si no hay conexión a internet, la pantalla indicará el modo offline con un ícono visible.

3.6 · Operación offline

ID	REQUERIMIENTO
RF-28	El ESP32 almacenará localmente el horario activo en memoria flash (SPIFFS / LittleFS).
RF-29	La app Flutter utilizará caché local para permitir consulta y edición de horarios sin conexión.
RF-30	Los cambios realizados offline se sincronizarán automáticamente con el backend al recuperar conectividad.
RF-31	El módulo ESP32 operará el timbre automático de forma autónoma aunque pierda conexión al servidor.

3.7 · Módulo de profesores y notificaciones de clase Nuevo en v2.0

ID	REQUERIMIENTO
RF-32	Cada profesor tendrá un perfil en el sistema con: nombre, correo, materia(s) que dicta y token FCM de su dispositivo.
RF-33	El administrador podrá crear, editar y eliminar perfiles de profesores desde la app.
RF-34	Cada profesor tendrá un horario semanal asignado con entradas que incluyen: día, hora de inicio, hora de fin, materia y salón.
RF-35	El administrador podrá cargar y modificar el horario semanal de cada profesor desde la app.
RF-36	Al activarse el timbre (automático, manual o sísmico), **Vapor** consultará qué clase corresponde a continuación para cada profesor según su horario en PostgreSQL.
RF-37	Se enviará una notificación push FCM personalizada a cada profesor con: hora de la próxima clase, materia y salón al que debe dirigirse.
RF-38	Si un profesor no tiene clase en el siguiente período, no recibirá notificación.
RF-39	Los profesores tendrán acceso a la app con rol Profesor, diferente al Administrador y al Operador.
RF-40	La app en modo Profesor mostrará el horario semanal completo del profesor autenticado.
RF-41	La clase en curso se resaltará visualmente con un indicador de color y etiqueta En curso.
RF-42	La próxima clase se resaltará con un indicador secundario y etiqueta Próxima.
RF-43	El horario del profesor se actualizará en tiempo real si el administrador realiza cambios durante el día.

Requerimientos no funcionales

ID	REQUERIMIENTO
RNF-01	La app usará los colores institucionales del colegio como paleta principal.
RNF-02	La interfaz seguirá los lineamientos de Material Design 3.
RNF-03	La app debe ser compatible con Android 8.0+ e iOS 14+.
RNF-04	El tiempo de respuesta entre comando de la app y activación del timbre físico no debe superar 2 segundos con conexión estable.
RNF-05	El envío de notificaciones FCM a todos los profesores debe completarse en menos de 5 segundos desde la activación del timbre.
RNF-06	Toda comunicación entre la app y el backend usará HTTPS con certificado SSL válido.
RNF-07	Las contraseñas se almacenarán en PostgreSQL con hash bcrypt. Nunca en texto plano.
RNF-08	La autenticación se realizará mediante JWT con tiempo de expiración configurable.
RNF-09	El código Flutter seguirá arquitectura Clean Architecture con separación por capas (data / domain / presentation).

05

Stack tecnológico

App móvil

Framework	`Flutter (Dart)`
State management	`Riverpod / Bloc`
Caché local	`Hive / Isar`
Autenticación	`JWT almacenado en secure storage`
Notificaciones	`flutter_local_notifications + FCM`

Backend

Framework	`Vapor (Swift)`
Base de datos	`PostgreSQL`
ORM	`Fluent (fluent-postgres-driver)`
Autenticación	`JWT con vapor/jwt + Bcrypt`
Servidor	`AlmaLinux — Nginx reverse proxy`
Dominio	`host.swiftydevs.dev`

Notificaciones push

Servicio	`FCM — Firebase Cloud Messaging (solo canal de entrega)`
Integración	`Vapor llama directamente a la API HTTP de FCM`
Dependencia Firebase	`Ninguna — solo se usa la Server Key de FCM`

Módulo físico

Microcontrolador	`ESP32 DevKit v1 / v4`
Sensor sísmico	`MPU-6050 (acelerómetro + giroscopio)`
Pantalla	`OLED 1.3" I2C (SH1106)`
Actuador	`Relé de estado sólido 3.3V`
Comunicación	`HTTP REST hacia Vapor vía WiFi`

Plan de desarrollo por fases

FASE	ALCANCE
Fase 1 — Base del proyecto Mes 1	Proyecto Flutter + Vapor configurados · PostgreSQL con modelo de datos · JWT Auth funcionando · Los 3 roles operativos.
Fase 2 — Módulo de timbre Mes 2	Sincronización ESP32 con Vapor · Timbre automático y manual · Operación offline · Pantalla OLED funcionando.
Fase 3 – 4 — Módulo de profesores Mes 3 - 4	CRUD de perfiles y horarios de profesores · Vista semanal con clase en curso · Notificaciones FCM desde Vapor.
Fase 5 — Módulos avanzados Mes 5	Horarios especiales con date picker · Módulo sísmico con algoritmo anti-vehicular · Historial de eventos.
Fase 6 — Pulido y entrega Mes 6	UI con colores institucionales · Pruebas reales con ESP32 físico · Corrección de bugs · Documentación final.

07

Entregables esperados

01 · App Flutter (Android + iOS)

Módulos de usuarios, horarios, timbre manual y vista de profesores.

02 · Backend Vapor

API REST documentada con todos los endpoints del sistema.

03 · Base de datos PostgreSQL

Modelo relacional completo con migraciones y datos iniciales.

04 · Firmware ESP32

Código fuente documentado para el módulo físico.

05 · Manual de usuario

Guía básica para el administrador (Doña Andrea).

06 · Diagrama de flujo

Algoritmo anti-vibración vehicular para el módulo sísmico.

Sistema de Timbre Escolar Inteligente.

Swifty.Devs

DOCUMENTO DE REQUERIMIENTOS V4.0 · DOSQUEBRADAS, COLOMBIA · 2026

BUILT FOR APPLE. DAY 0.